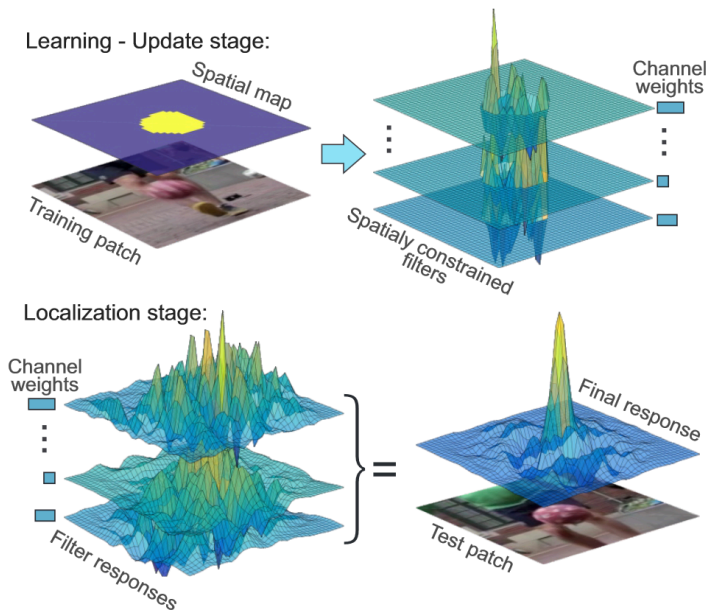
Overview

The DCF Tracker is an online visual tracker that employs a Discriminative Correlation Filter for visual object tracking. It learns a object-specific correlation filter and localizes the same object in the next frames based on what was learned. For more information, see the CSR-DCF paper [1].

Consult the **DCF Tracker sample application** for an example of how a simple DCF Tracker can be implemented.

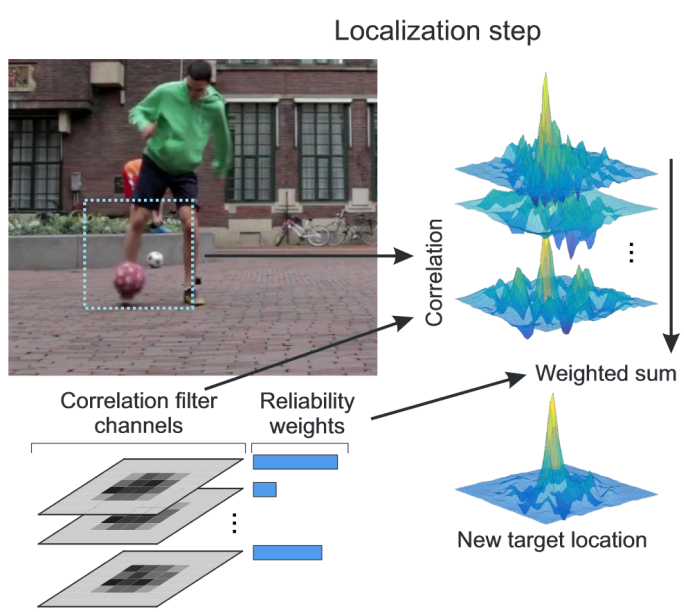
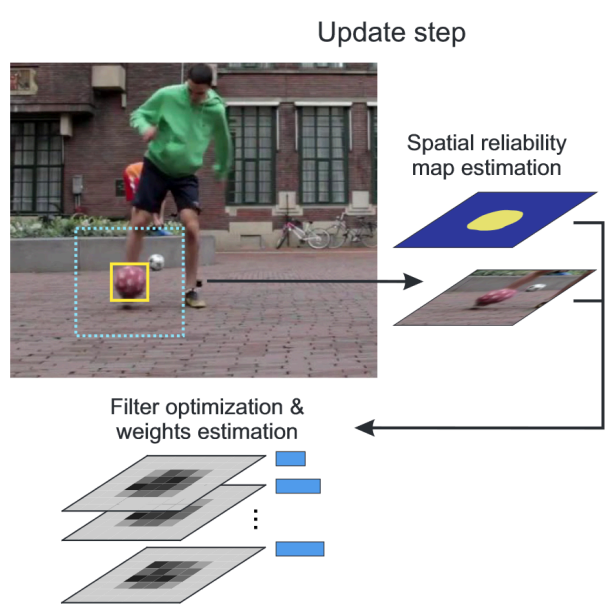
Note

VPI implements a subset of the CSR-DCF paper. It doesn't implement the discriminative scale-space tracking, hence not tracking object's scale changes over time. Only the object's position is tracked.



Source: CSR-DCF paper [1]

Implementation

Localize Object	Update Filters
 Source: CSR-DCF paper [1]	

Definitions

Object Patches: detected objects that are cropped from the input frame using its bounding box and then rescaled into a small square image are referred to as object patches. The patches of all tracked objects are stored in a tall contiguous image, one on top of each other. Patch extraction and scaling can be performed by **Crop Scaler** .

Feature Patches: dense multi-dimensional features extracted from the object patches. Each dimensions accounts for a particular feature, represented by a one-channel image plane. Each plane consists of rectangular regions stacked on top of each other, each region corresponding to one object patch, in the same order. All planes have the same size, which is given by the `featurePatchSize` DCFTracker parameter.

The DCFTracker operates on the tracked objects in two distinct phases: localization and update. In localization phase the tracked objects are detected in the input frames and their bounding boxes are updated to reflect their estimated position. In the update phase, the internal object model is updated given its current estimated bounding box.

Localize objects

In localization phase the tracked objects are detected in the input frames and their bounding boxes are updated to reflect their estimated position.

The function operates on the input object array, `inObjects` and their corresponding image patches in `inPatches`. The function does not update the `inObjects` array in-place. Instead, it gets copied to `outObjects`, but with the updated bounding box. By not doing an in-place operation, both previous and new bounding boxes are preserved. These could be useful for further refinement of the bounding boxes, external to DCFTracker. If in-place object array update is desired, `inObjects` can also be passed as output array.

Depending on the object's state, the following operations are performed:

- NEW: does nothing.
- TRACKED: localize the object on its corresponding image patch and estimate its new bounding box on it, writing the result to the output.
- SHADOW TRACKED: same as TRACKED.
- LOST: does nothing.

Additionally, the calculated correlation responses can be returned if user passes a valid **VPIImage** as the `outCorrelationResponses` parameter. It could be used by the application to decide on the fate of the tracked object, that is, whether tracking was lost or not. This decision is then reflected on the new object state. If it's decided that the tracking was lost, the object's state must be set to LOST.

The `outCorrelationResponse` image contains the correlation response of all objects being handled, no matter the object's state. The image format is **VPI_IMAGE_FORMAT_F32**. Only objects with state TRACKED or SHADOW_TRACKED have valid correlation response. The image is a vertical strip of square patches, each one having width equal to `featureImgSize`.

If needed, this image must be allocated by the user with the correct dimensions and format. It must be tall enough to contain at least `maxTargets`, as specified during DCFTracker payload creation.

Update Correlation filters

The update phase is where new objects can be added, removed, and the update of internal model of the tracked objects is done, given its currently estimated bounding box. Depending on the object's state, the following operations are performed:

- NEW: objects are added to the DCFTracker instance. Features are extracted from objects' image patches, window masking is applied, and its filter weight is calculated and stored internally. All existing data associated with a previous object is overwritten. Tracking for this new object starts from scratch.
- TRACKED: Just like NEW, but since object is already being tracked, there's no need to initialize internal model, it only gets updated using the given object patch.
- SHADOW TRACKED: does nothing.
- LOST: does nothing.

Once a correlation filter is generated for a target, the DCF tracker employs an exponential moving average for temporal consistency, when the optimal correlation filter is created and updated over consecutive frames. The learning rate for this moving average can be configured by `filterLr` and `filterChannelWeightsLr` for the correlation filters and their channel weights, respectively. The standard deviation for Gaussian for the desired response used when creating an optimal DCF filter can also be configured by `gaussianSigma`.

C API functions

For list of limitations, constraints and backends that implements the algorithm, consult reference documentation of the following functions:

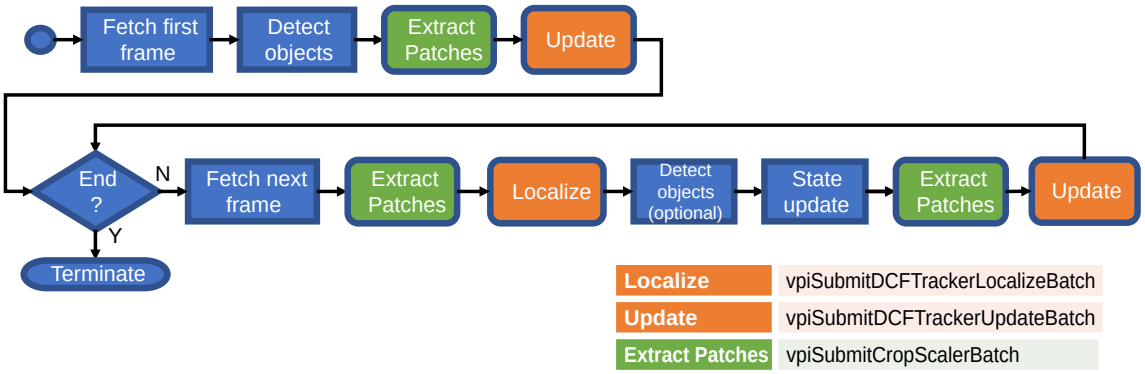
Function	Description
vpiInitDCFTrackerCreationParams	Initialize **VPIDCFTrackerCreationParams** with default values.
vpiCreateDCFTracker	Creates payload for DCF Tracker.

vpiInitDCFTrackerParams	Initialize VPIDCFTrackerParams with default values.
vpiSubmitDCFTrackerLocalizeBatch	Localizes each tracked object in the input image patches using the Discriminative Correlation Filter method.
vpiSubmitDCFTrackerUpdateBatch	Update internal object tracking information based on its state and its corresponding input image patch.
vpiDCFTrackerGetChannelWeights	Returns the array with channel weight information for each tracked object.

Usage

Building a DCF Tracker pipeline

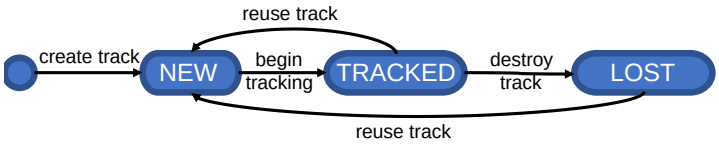
The DCF components described above can be assembled to make a tracker in various ways. One common way is as shown below. Initial frame and reference bounding boxes (typically from a detector) are used to update (vpiSubmitDCFTrackerUpdateBatch), i.e. learn a set of discriminative correlation filters and corresponding channel weights. Subsequent frames are fed into a loop which alternates vpiSubmitDCFTrackerLocalizeBatch and vpiSubmitDCFTrackerUpdateBatch steps to constantly localize the object and update correlation filters based on the new bounding box locations of the target object.



State Transition Management

State transition management must be handled by the user. A sample transition map is shown below. In a typical scenario, after frames are processed they can be clasified into the following states:

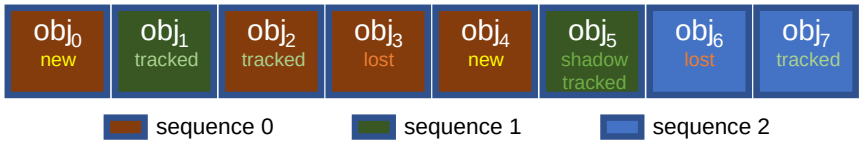
1. existing objects that are being tracked (VPI_TRACKING_STATE_TRACKED)
2. new objects that were added by a detector mechanism (VPI_TRACKING_STATE_NEW)
3. objects whose tracking was lost (VPI_TRACKING_STATE_LOST). Objects that are labelled as VPI_TRACKING_STATE_LOST are removed. Tracking states such as VPI_TRACKING_STATE_SHADOW_TRACKED are supported by the API. This is an optional tracker state that may find use in advanced state transition management.



The state transitions are triggered by the user by explicitly updating the object's state in the object array. This grants the user total control of the object's lifetime and what kind of processing is done on the objects.

Object handling

All objects being handled are stored in a single VPIArray object. The element type of the object array is VPI_ARRAY_TYPE_DCF_TRACKED_BOUNDING_BOX . The position of an object in the array is fixed during the object's lifetime. They are processed independently and can come from multiple input sequences, as shown in the figure below.



The user is responsible for managing the object array. Objects whose tracking was lost (or any object for that matter) can be reused for new objects when needed, just set its state to NEW, and initialize the rest of object attributes. If tracking of objects at the end of the array was lost, the whole array size can be decreased accordingly, thereby improving algorithm runtime performance.

Any additional attribute associated with the object can be managed externally. They attributes can be associated with the corresponding object managed by DCFTracker via its index in the object array, or by using the `userData` pointer in the **VPIDCFTrackedBoundingBox** structure.

Configuring a DCF Tracker pipeline

There are two sets of configuration parameters available in DCFTracker. The first set corresponds to parameters that affect the allocation of internal resources needed. These are defined when creating the DCFTracker instance and can't be modified afterwards. These are the **VPIDCFTrackerCreationParams** and **VPIDCFTrackerCreationFlag** .

The second set corresponds to parameters that can be modified at runtime, e.g., after every frame or as per user's need. **VPIDCFTrackerParams** defines these parameters.

These structures can be filled with default values by using the initialization functions **vpiInitDCFTrackerCreationParams** and **vpiInitDCFTrackerParams**.

For more information, see **DCF Tracker** in the "C API Reference" section of *VPI - Vision Programming Interface*.

For a complete multi-object tracking solution, please refer to the NvDCF multi-object tracker in NVIDIA DeepStream SDK.

References

1. Alan Lukezic, Tomas Vojir, Luka Cehovin Zajc, Jiri Matas, Matej Kristan, "Discriminative Correlation Filter with Channel and Spatial Reliability". Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

Performance

For information on how to use the performance tables below, see **Algorithm Performance Tables**.
Before comparing measurements, consult **Comparing Algorithm Elapsed Times**.
For further information on how performance was benchmarked, see **Performance Benchmark**.

clear filters

Device: Jetson AGX Thor - **Streams:** 1

operation	format	patch size	HOG cell size	flags	#sequences	#objs	CUDA	PVA
update	rgba8	18	4	colornames,HOG	32	64	0.2±0.2 ms	n/a
update	rgba8	24	4	colornames,HOG	32	64	0.26±0.07 ms	n/a
update	rgb8p	24	4	colornames,HOG	32	64	n/a	4.084±0.006 ms
update	rgba8	32	4	colornames,HOG	32	64	0.37±0.01 ms	n/a
update	rgba8	24	4	colornames,HOG	32	64	0.21±0.07 ms	n/a
update	rgb8p	24	4	colornames,HOG	32	64	n/a	4.090±0.003 ms
update	rgba8	24	8	colornames,HOG	32	64	0.46±0.01 ms	n/a
update	rgba8	24	16	colornames,HOG	32	64	1.60±0.01 ms	n/a
update	rgba8	24	4	colornames,HOG	32	64	0.20±0.04 ms	n/a
update	rgb8p	24	4	colornames,HOG	32	64	n/a	4.080±0.003 ms
update	rgba8	24	4	colornames,HOG	32	512	1.820±0.005 ms	n/a
update	rgb8p	24	4	colornames,HOG	32	512	n/a	30.31±0.05 ms
update	rgba8	24	4	colornames,HOG	16	64	0.24±0.08 ms	n/a
update	rgb8p	24	4	colornames,HOG	16	64	n/a	4.085±0.003 ms
update	rgba8	24	4	colornames,HOG	32	64	0.24±0.07 ms	n/a
update	rgb8p	24	4	colornames,HOG	32	64	n/a	4.083±0.005 ms
update	rgba8	24	4	HOG	32	64	0.162±0.004 ms	n/a
update	rgba8	24	4	colornames	32	64	0.1±0.2 ms	n/a
localize	rgba8	18	4	colornames,HOG	32	64	0.1±0.2 ms	n/a
localize	rgba8	24	4	colornames,HOG	32	64	0.14±0.01 ms	n/a
localize	rgb8p	24	4	colornames,HOG	32	64	n/a	2.675±0.006 ms
localize	rgba8	32	4	colornames,HOG	32	64	0.23±0.09 ms	n/a
localize	rgba8	24	4	colornames,HOG	32	64	0.2±0.2 ms	n/a
localize	rgb8p	24	4	colornames,HOG	32	64	n/a	2.673±0.005 ms
localize	rgba8	24	8	colornames,HOG	32	64	0.38±0.02 ms	n/a
localize	rgba8	24	16	colornames,HOG	32	64	1.54±0.01 ms	n/a
localize	rgba8	24	4	colornames,HOG	32	64	0.2±0.2 ms	n/a
localize	rgb8p	24	4	colornames,HOG	32	64	n/a	2.675±0.005 ms
localize	rgba8	24	4	colornames,HOG	32	512	1.282±0.006 ms	n/a
localize	rgb8p	24	4	colornames,HOG	32	512	n/a	19.04±0.02 ms
localize	rgba8	24	4	colornames,HOG	16	64	0.2±0.2 ms	n/a
localize	rgb8p	24	4	colornames,HOG	16	64	n/a	2.680±0.005 ms
localize	rgba8	24	4	colornames,HOG	32	64	0.2±0.2 ms	n/a
localize	rgb8p	24	4	colornames,HOG	32	64	n/a	2.672±0.005 ms
localize	rgba8	24	4	HOG	32	64	0.2±0.2 ms	n/a
localize	rgba8	24	4	colornames	32	64	0.06±0.05 ms	n/a

Corporate Info

NVIDIA.com HOME
About NVIDIA

NVIDIA Developer

Developer Home
Blog

Resources

Contact Us
Developer Program

Privacy Policy | Manage My Privacy | Do Not Sell or Share My Data | Terms of Service | Accessibility | Corporate Policies | Product Security | Contact

Copyright © 2026 NVIDIA Corporation